

# MRRpy: A modular Python model for distributed rainfall–runoff generation and flood routing

Saurav Bhattarai<sup>1</sup>, Nawa Raj Pradhan<sup>2</sup>, and Rocky Talchabhadel<sup>1</sup>

<sup>1</sup> Department of Civil and Environmental Engineering, Jackson State University, Jackson, MS, USA <sup>2</sup> Coastal and Hydraulics Laboratory, U.S. Army Engineer Research and Development Center, Vicksburg, MS, USA ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

## Software

- Review
- Repository
- Archive

Editor: [Open Journals](#)

## Reviewers:

- @openjournals

Submitted: 01 January 1970

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

## Summary

MRRpy is an open-source Python package for distributed, event-scale rainfall-runoff simulation and channel flood routing. Given a digital elevation model, it delineates a D8 drainage network, generates surface runoff by one of several interchangeable methods, and routes the resulting flow to a basin outlet using one of five interchangeable numerical schemes: kinematic wave, explicit diffusive wave, variable-parameter Muskingum–Cunge, local-inertial dynamic wave, and a semi-implicit diffusive wave. The semi-implicit scheme takes advantage of a simple property of D8 networks. Because a filled DEM's flow directions form a tree rather than a general graph, the linearized flow equations can be solved exactly, in time linear in the number of cells, without the Courant restriction that limits explicit schemes on steep terrain.

Runoff generation, channel routing, upstream boundary inflow, and diagnostic output are independent stages of one pipeline, driven by a single configuration object. Changing one field of that configuration is enough to swap a runoff method, a routing scheme, or the source of soil and rainfall forcing, which makes it straightforward both to isolate the effect of a single modeling decision and to run many configurations in sequence for calibration or uncertainty exploration. Every routing run produces an outlet hydrograph and a mass-balance ledger; per-cell depth, velocity, and discharge fields, and fixed-point gauge series, are recorded on request. An upstream discharge hydrograph, from a gauge, a reservoir, or another model, can also be injected directly into the channel network, with or without rainfall. The same configuration and pipeline are exposed through a Python API, a command-line interface, and a QGIS plugin, and can draw rainfall, soil, and terrain data from Google Earth Engine when no local data exist for a basin.

## Statement of need

Isolating a single modeling decision in an event-scale flood study is harder than it should be. A hydrologist who wants to know whether Green–Ampt infiltration-excess or a saturation-excess variable source area produces a better hydrograph, with routing held fixed, usually has to patch together two independent research codes or rebuild the runoff-to-routing-to-diagnostics plumbing by hand. The reverse comparison, holding runoff fixed while testing routing numerics, is just as common and just as tedious to set up. Consistent spatial inputs, boundary handling, mass-balance accounting, and output formats are exactly the parts of that comparison that are easy to get subtly wrong and expensive to rebuild for every new experiment. MRRpy factors runoff generation and channel routing into independent, registrable pipeline stages behind one configuration object, so changing the comparison means changing one setting rather than one codebase.

Because every run is driven by the same configuration object, the same design supports a second,

equally important use: running many configurations instead of one. A parameter sweep, or a full factorial across routing scheme, runoff mechanism, and channel geometry, is simply a loop over configuration values and pipeline calls, and the mass-balance ledger accumulates one row per run so that dozens or hundreds of configurations stay directly comparable in a single table instead of being scattered across separate output folders. This is not a hypothetical capability: the repository's own research tooling already uses this pattern to run full-factorial studies of several hundred configurations across multiple flood events, resumable across sessions and organized into a self-describing folder tree. The same optional GPU backend that accelerates one run also accelerates a sweep of many runs without any change to the model code, so a small calibration exercise can stay on a laptop CPU while a larger one moves to a GPU by changing a single configuration field.

That flexibility matters most when time is short. A flood emergency in a basin with no rain gauges, no soil survey, and no existing hydrologic model is common rather than rare. Because MRRpy can resolve rainfall, soil, and terrain forcing directly from Google Earth Engine given only a watershed boundary, a working model of an ungauged basin can be assembled without a separate data-acquisition and GIS-preprocessing step. Because the runoff and routing choices are configuration values rather than code, the same setup can then be run across a plausible range of infiltration, source-area, and routing assumptions instead of committing to a single untested guess. The result is a bounded set of hydrographs rather than one, produced fast enough to inform a response rather than a postmortem.

MRRpy serves environmental engineers and hydrologists running controlled method comparisons or calibration sweeps, researchers coupling routing-only runs to an external rainfall-runoff or reservoir model through the boundary-condition interface, and instructors and students who need a runnable, inspectable implementation of each method rather than a black box. The same configuration and pipeline are exposed through a Python API, a command-line interface, and a QGIS desktop plugin, so the package is usable by a GIS practitioner as well as a programmer. An interactive textbook bundled with the repository works through the physical assumptions behind each modeling choice.

## State of the field

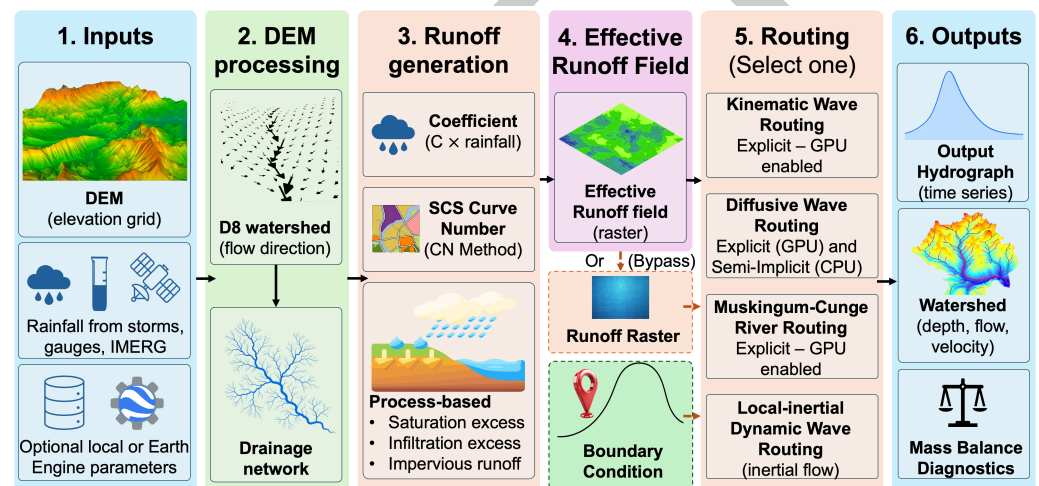
pysheds (Bartos, 2020) and pyflwdir (Eilander & Boisgontier, 2021) are strong Python libraries for D8 terrain and flow-network analysis, but they stop at delineation and flow directions. MRRpy uses either as its delineation engine and continues through runoff generation, routing, and diagnostics. Landlab (Barnhart et al., 2020) is a general-purpose Earth-surface-dynamics component framework capable of far more than event flood routing, though assembling a model from its lower-level components takes more work than MRRpy's single configuration file, since MRRpy is purpose-built for the narrower task of comparing runoff methods and routing schemes. GSSHA/CASC2D (Downer & Ogden, 2004) is a mature, physically comprehensive distributed model, and MRRpy's saturation-excess mechanism implements the same Pradhan and Ogden variable source area formulation (Pradhan & Ogden, 2010) published by one of GSSHA's own authors. GSSHA is nonetheless a compiled model conventionally driven through a separate desktop GUI, not an installable, scriptable Python package with a registrable-mode API.

LISFLOOD-FP (Bates et al., 2010) and its local-inertial formulation (Almeida et al., 2012) solve the fuller two-dimensional shallow-water problem needed for floodplain inundation extent. MRRpy solves a cheaper, narrower problem: one-dimensional flow on the D8 channel network rather than inundation extent, with the same local-inertial equations available as one routing choice among five rather than as its central contribution. Its semi-implicit diffusive-wave scheme is best understood as a lightweight, open, Python-native counterpart to the semi-implicit solvers used in tools such as HEC-RAS (Brunner, 2021). Because a D8-conditioned network is provably a tree rather than a general graph, the same linearized-diffusion system that such tools solve iteratively can instead be solved exactly, by one forward-and-back sweep in time

linear in the cell count. This simplification is specific to tree-structured networks and would not carry over to a general cross-section network with junctions and loops. MRRpy does not claim to replace a multidimensional floodplain-inundation solver or a general river-network hydraulics package. Packaging a focused set of runoff and routing choices behind one configuration, with consistent forcing, boundary handling, and diagnostics, is a more tractable contribution than extending a terrain library or a general framework into a complete hydrologic model.

## Software design

The architecture in Figure 1 separates inputs, DEM processing, runoff generation, routing, and outputs. A Config object drives the same run\_pipeline orchestrator through Python, the CLI, or QGIS. DEM processing reprojects and conditions terrain, computes D8 flow directions and accumulation, and delineates the catchment to a selected outlet. Users can choose pysheds (Bartos, 2020; O’Callaghan & Mark, 1984) or pyflwdir (Eilander & Boisgontier, 2021; Wang & Liu, 2006).



**Figure 1:** MRRpy’s processing chain, from terrain and precipitation inputs through D8 watershed delineation to selectable runoff-generation and routing methods, ending in an outlet hydrograph, watershed depth/flow/velocity fields, and a mass-balance diagnostic. Runoff generation offers a spatial coefficient, SCS Curve Number, or the process-based combination of saturation-excess, infiltration-excess, and impervious mechanisms; the resulting effective-runoff field, a prescribed runoff raster, or an external point-inflow boundary condition then drives one of four routing families: kinematic, diffusive (explicit or semi-implicit), Muskingum–Cunge, and local-inertial dynamic wave. Virtual gauges and per-cell field archives are optional and not shown. The direct-rainfall runoff mode is omitted from the schematic for clarity.

Precipitation can be a uniform event, a gauge field interpolated by Thiessen polygons or inverse-distance weighting, or IMERG satellite rainfall (Huffman et al., 2020). An optional elevation rule excludes snow-zone precipitation from event runoff. Soil and land-cover parameters can come from local inputs or Google Earth Engine (Gorelick et al., 2017): root-zone depth from ESA WorldCover land cover (Zanaga et al., 2022), field capacity and wilting point from SoilGrids (Poggio et al., 2021), and porosity and saturated hydraulic conductivity from HiHydroSoil (Simons et al., 2020).

The runoff layer converts rainfall to effective runoff in metres per second per active cell. Its built-in choices are direct rainfall, a spatial runoff coefficient, SCS Curve Number (Chow et al., 1988; Jaafar et al., 2019), a prescribed raster time series, and a process-based mode. The latter combines Variable Source Area saturation-excess (Pradhan & Ogden, 2010), Green–Ampt infiltration-excess (Green & Ampt, 1911; Rawls et al., 1983), and impervious shedding without

118 counting the same rainfall twice. The RunoffMode interface exposes `get_effective_1d` and  
 119 `update_state`; a registration decorator allows new methods, although the fixed-choice Config  
 120 list must also be extended to use a new name through the standard configuration path.

121 The router receives the same effective-runoff field regardless of its source. It offers kinematic  
 122 wave, explicit diffusive wave, variable-parameter Muskingum–Cunge (Cunge, 1969; Ponce &  
 123 Yevjevich, 1978), local-inertial dynamic wave (Almeida et al., 2012; Bates et al., 2010), and  
 124 semi-implicit diffusive wave schemes. Explicit storage-based branches can use adaptive Courant  
 125 steps and a volume-conservative flux limiter; Muskingum–Cunge instead advances discharge as  
 126 its state. The semi-implicit branch iterates a water-surface conductance and solves a coupled  
 127 linear system on the D8 tree. Its direct tree sweep costs linear time per iteration, with optional  
 128 Numba (Lam et al., 2015) acceleration and a SciPy (Virtanen et al., 2020) sparse fallback. It  
 129 runs on the CPU; the other routing paths can use an optional CuPy GPU backend. The linear  
 130 solve removes the explicit diffusive scheme's CFL stability restriction, but time-step size and  
 131 nonlinear convergence still require attention. Adding a new routing scheme requires both a  
 132 registry entry and integration with the time loop.

133 An upstream discharge hydrograph can be injected into the routing network independently  
 134 of rainfall-runoff generation. A boundary-condition entry locates the injection point by row  
 135 and column, by projected easting and northing, or by latitude and longitude, snaps it to the  
 136 nearest channel cell (the cell of highest flow accumulation within a configurable radius), and  
 137 reads a discharge time series from a CSV file. The value is linearly interpolated in time and  
 138 holds at the last recorded value beyond the series, with a warning printed to the log. Multiple  
 139 boundary points are supported and summed where two of them snap to the same cell. Setting  
 140 rainfall to zero then yields a pure boundary-driven routing run, the mechanism for routing  
 141 an upstream gauge record, a reservoir release schedule, or another model's discharge output  
 142 through the channel network without invoking runoff generation at all.

143 Every routing run writes an outlet hydrograph CSV, reporting peak discharge and time to  
 144 peak, and appends one row to a routed-water mass-balance ledger recording rainfall input,  
 145 boundary inflow, outflow, storage change, and closure error. That ledger accumulates across  
 146 successive runs rather than being overwritten, so a parameter sweep remains self-describing  
 147 and directly comparable in a single table. A full pipeline run additionally writes the watershed  
 148 files produced by DEM processing. Two further outputs are optional. Named virtual gauges  
 149 record depth, discharge, and velocity at fixed, channel-snapped locations at every output  
 150 step, a memory-cheap way to obtain fine-cadence series at specific points without saving the  
 151 whole domain. A field archive records per-cell depth, velocity, discharge, and volume at a  
 152 configurable stride, into a compact, georeferenced archive that a loader scatters back onto  
 153 the two-dimensional grid for plotting or animation. Process-based runoff runs additionally  
 154 partition the outlet volume by generating mechanism (saturation-excess, infiltration-excess,  
 155 impervious). A bundled plotting module provides one-call helpers for the hydrograph, a raster,  
 156 the watershed boundary, and the mass-balance ledger, each accepting a file path, a DataFrame,  
 157 a pipeline result, or the Config object itself, so a run's results can be inspected without  
 158 hand-written matplotlib or rasterio code. These common outputs make comparisons across  
 159 configurations inspectable, without implying that different schemes produce identical physical  
 160 solutions.

## 161 Research impact statement

162 MRRpy's exact, linear-time tree solve for the semi-implicit diffusive-wave scheme is, on its  
 163 own, a transferable numerical-methods contribution. It removes a stability restriction that  
 164 has historically forced a choice between small time steps and an iterative implicit solve, by  
 165 exploiting a structural property that other D8-based grid routers already have but do not  
 166 currently use this way: D8 flow direction is a tree, not a general graph. This claim is verified  
 167 rather than asserted. A dedicated verification script compares the tree solve against dense  
 168 and sparse reference linear solvers, checks mass-balance closure on a synthetic valley DEM,

169 confirms convergence to the Manning normal-depth solution in the kinematic limit, reproduces  
170 the expected diffusion-wave attenuation and lag signature, and compares the routed response  
171 against the analytical Hayami linear diffusion-wave benchmark. The scheme is also exercised  
172 inside the end-to-end mass-balance closure test that continuous integration runs across Python  
173 3.9 through 3.12.

174 Accessibility is a second channel of impact. Exposing one configuration and pipeline through a  
175 Python API, a command-line interface, and a QGIS desktop plugin means the same runoff  
176 and routing choices are usable by a programmer-researcher and by a GIS-trained practitioner  
177 or agency hydrologist, without either needing the other's tooling. Optional Google Earth  
178 Engine integration removes the largest practical barrier to an event study on an ungauged  
179 or data-sparse basin, the local acquisition and preprocessing of satellite rainfall, soil, and  
180 terrain rasters, reducing that step to specifying a watershed boundary. Combined with the  
181 configuration-driven sweep pattern described above, this makes rapid, multi-scenario model  
182 setup practical for basins and situations, including time-critical ones, where no dedicated  
183 hydrologic model previously existed.

184 The repository includes an end-to-end Trishuli basin example in Nepal, driven by satellite rainfall,  
185 and a separate upstream-inflow boundary-condition scenario, together with an interactive  
186 textbook that works through the physical assumptions behind each runoff and routing choice,  
187 including the graph-theoretic argument for the semi-implicit scheme's exact solve. These  
188 are reproducible use cases and teaching material, not claims of predictive validation against  
189 observed streamflow. Automated tests separately cover configuration round-tripping, runoff-  
190 mechanism composition, raster alignment, boundary inflow, and end-to-end mass-balance  
191 closure across all five routing schemes. Together, these materials support controlled method  
192 comparisons, give researchers a starting point for independent evaluation on their own basins,  
193 and give instructors a runnable reference implementation rather than a black box.

## 194 AI usage disclosure

195 Generative AI assistance, Anthropic's Claude Code CLI using several Claude model versions  
196 (Sonnet 4.6, Opus 4.6, Opus 4.8, Sonnet 5, and Fable 5) over the course of development,  
197 was used throughout this project for code implementation, refactoring, test scaffolding, and  
198 documentation drafting, and was used to draft and revise text in this manuscript. All AI-assisted  
199 code and text were reviewed, tested, and edited by the human authors, who are responsible for  
200 the correctness of the software, the validity of the reported results, and the content of this  
201 paper.

## 202 Acknowledgements

203 MRRpy is built on the open-source scientific-Python and geospatial ecosystem, including NumPy  
204 (Harris et al., 2020), SciPy (Virtanen et al., 2020), rasterio, GeoPandas, pysheds, and  
205 pyflwdir, and we thank their developers and maintainers.

206 This research was supported in part by an appointment to the Department of Defense (DOD)  
207 Research Participation Program administered by the Oak Ridge Institute for Science and  
208 Education (ORISE) through an interagency agreement between the U.S. Department of  
209 Energy (DOE) and the DOD. ORISE is managed by ORAU under DOE contract number  
210 DE-SC0014664. All opinions expressed in this paper are the authors' and do not necessarily  
211 reflect the policies and views of DOD, DOE, or ORAU/ORISE.

212 This research is also supported by the Hydrological Impacts Computing, Outreach, and Resiliency  
213 Partnership (HICORPS) Project, developed in collaboration with the U.S. Army Engineer  
214 Research and Development Center (ERDC), WOOLPERT, and Taylor Engineering. We thank

215 Jackson State University (JSU) for providing computational resources and infrastructure support  
216 throughout this research.

## 217 References

- 218 Almeida, G. A. M. de, Bates, P., Freer, J. E., & Souvignet, M. (2012). Improving the stability  
219 of a simple formulation of the shallow water equations for 2-D flood modeling. *Water*  
220 *Resources Research*, 48(5), W05528. <https://doi.org/10.1029/2011WR011570>
- 221 Barnhart, K. R., Hutton, E. W. H., Tucker, G. E., Gasparini, N. M., Istanbuluoglu, E., Hobley,  
222 D. E. J., Lyons, N. J., Mouchene, M., Nudurupati, S. S., Adams, J. M., & Bandaragoda, C.  
223 (2020). Short communication: Landlab v2.0: A software package for earth surface dynamics.  
224 *Earth Surface Dynamics*, 8(2), 379–397. <https://doi.org/10.5194/esurf-8-379-2020>
- 225 Bartos, M. (2020). *Pysheds: Simple and fast watershed delineation in python*. <https://doi.org/10.5281/zenodo.3822494>
- 226
- 227 Bates, P. D., Horritt, M. S., & Fewtrell, T. J. (2010). A simple inertial formulation of the  
228 shallow water equations for efficient two-dimensional flood inundation modelling. *Journal*  
229 *of Hydrology*, 387(1–2), 33–45. <https://doi.org/10.1016/j.jhydrol.2010.03.027>
- 230 Brunner, G. W. (2021). *HEC-RAS river analysis system hydraulic reference manual, version*  
231 *6.0*. US Army Corps of Engineers, Hydrologic Engineering Center.
- 232 Chow, V. T., Maidment, D. R., & Mays, L. W. (1988). *Applied hydrology*. McGraw-Hill.
- 233 Cunge, J. A. (1969). On the subject of a flood propagation computation method (muskingum  
234 method). *Journal of Hydraulic Research*, 7(2), 205–230. [https://doi.org/10.1080/](https://doi.org/10.1080/00221686909500264)  
235 [00221686909500264](https://doi.org/10.1080/00221686909500264)
- 236 Downer, C. W., & Ogden, F. L. (2004). GSSHA: Model to simulate diverse stream flow  
237 producing processes. *Journal of Hydrologic Engineering*, 9(3), 161–174. [https://doi.org/](https://doi.org/10.1061/(ASCE)1084-0699(2004)9:3(161))  
238 [10.1061/\(ASCE\)1084-0699\(2004\)9:3\(161\)](https://doi.org/10.1061/(ASCE)1084-0699(2004)9:3(161))
- 239 Eilander, D., & Boisgontier, H. (2021). *Pyflwdir: Fast methods to work with hydro- and*  
240 *topography data in pure python*. <https://github.com/Deltares/pyflwdir>
- 241 Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017).  
242 Google earth engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of*  
243 *Environment*, 202, 18–27. <https://doi.org/10.1016/j.rse.2017.06.031>
- 244 Green, W. H., & Ampt, G. A. (1911). Studies on soil physics: 1. The flow of air and water  
245 through soils. *The Journal of Agricultural Science*, 4(1), 1–24. [https://doi.org/10.1017/](https://doi.org/10.1017/S0021859600001441)  
246 [S0021859600001441](https://doi.org/10.1017/S0021859600001441)
- 247 Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D.,  
248 Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with  
249 NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- 250 Huffman, G. J., Bolvin, D. T., Braithwaite, D., Hsu, K.-L., Joyce, R., Kidd, C., Nelkin, E.  
251 J., Sorooshian, S., Tan, J., & Xie, P. (2020). *NASA global precipitation measurement*  
252 *(GPM) integrated multi-satellite retrievals for GPM (IMERG)* [Algorithm Theoretical Basis  
253 Document (ATBD), version 06]. NASA.
- 254 Jaafar, H. H., Ahmad, F. A., & El Beyrouthy, N. (2019). GCN250, new global gridded  
255 curve numbers for hydrologic modeling and design. *Scientific Data*, 6(1), 145. <https://doi.org/10.1038/s41597-019-0155-x>  
256
- 257 Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: A LLVM-based Python JIT compiler.  
258 *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 1–6.  
259 <https://doi.org/10.1145/2833157.2833162>

- O'Callaghan, J. F., & Mark, D. M. (1984). The extraction of drainage networks from digital elevation data. *Computer Vision, Graphics, and Image Processing*, 28(3), 323–344. [https://doi.org/10.1016/S0734-189X\(84\)80011-0](https://doi.org/10.1016/S0734-189X(84)80011-0)
- Poggio, L., Sousa, L. M. de, Batjes, N. H., Heuvelink, G. B. M., Kempen, B., Ribeiro, E., & Rossiter, D. (2021). SoilGrids 2.0: Producing soil information for the globe with quantified spatial uncertainty. *SOIL*, 7(1), 217–240. <https://doi.org/10.5194/soil-7-217-2021>
- Ponce, V. M., & Yevjevich, V. (1978). Muskingum-cunge method with variable parameters. *Journal of the Hydraulics Division*, 104(12), 1663–1667. <https://doi.org/10.1061/JYCEAJ.0005186>
- Pradhan, N. R., & Ogden, F. L. (2010). Development of a one-parameter variable source area runoff model for ungauged basins. *Advances in Water Resources*, 33(5), 572–584. <https://doi.org/10.1016/j.advwatres.2010.03.002>
- Rawls, W. J., Brakensiek, D. L., & Miller, N. (1983). Green-ampt infiltration parameters from soils data. *Journal of Hydraulic Engineering*, 109(1), 62–70. [https://doi.org/10.1061/\(ASCE\)0733-9429\(1983\)109:1\(62\)](https://doi.org/10.1061/(ASCE)0733-9429(1983)109:1(62))
- Simons, G. W. H., Koster, R., & Droogers, P. (2020). *HiHydroSoil v2.0 – a high resolution soil map of global hydraulic properties* (FutureWater Report No. 213). FutureWater. <https://www.futurewater.nl/wp-content/uploads/2020/10/HiHydroSoil-v2.0-High-Resolution-Soil-Maps-of-Global-Hydraulic-Properties.pdf>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- Wang, L., & Liu, H. (2006). An efficient method for identifying and filling surface depressions in digital elevation models for hydrologic analysis and modelling. *International Journal of Geographical Information Science*, 20(2), 193–213. <https://doi.org/10.1080/13658810500433453>
- Zanaga, D., Van De Kerchove, R., Daems, D., De Keersmaecker, W., Brockmann, C., Kirches, G., Wevers, J., Cartus, O., Santoro, M., Fritz, S., Lesiv, M., Herold, M., Tsendbazar, N.-E., Xu, P., Ramoino, F., & Arino, O. (2022). *ESA WorldCover 10 m 2021 v200*. <https://doi.org/10.5281/zenodo.7254221>